ENTERPRISE AI OPERATIONS COPILOT - PROJECT DOCUMENTATION

==========

1. PROJECT OVERVIEW

Enterprise AI Operations Copilot is a local AI-powered assistant designed to help employees and operations teams ask operational and document-based questions from one chat interface.

The system supports PDF document Q&A, employee information lookup, leave balance checking, date/time queries, calculations, general AI questions, conversation memory, and backend debugging.

The project combines Streamlit, FastAPI, LangGraph, MCP-style tools, SQLite, FAISS, BM25 retrieval, MiniLM embeddings, and Groq LLM responses.

2. PURPOSE OF THE PROJECT

The purpose of this project is to create a single AI copilot for day-to-day enterprise operations.

A user can:
- Upload company policies, handbooks, resumes, or other PDF files.
- Ask questions about the selected document.
- Get answers with source and page references.
- Ask employee-related questions.
- Check leave balance.
- Ask date/time questions.
- Perform simple calculations.
- Ask general AI questions.
- View backend reasoning and debug traces.

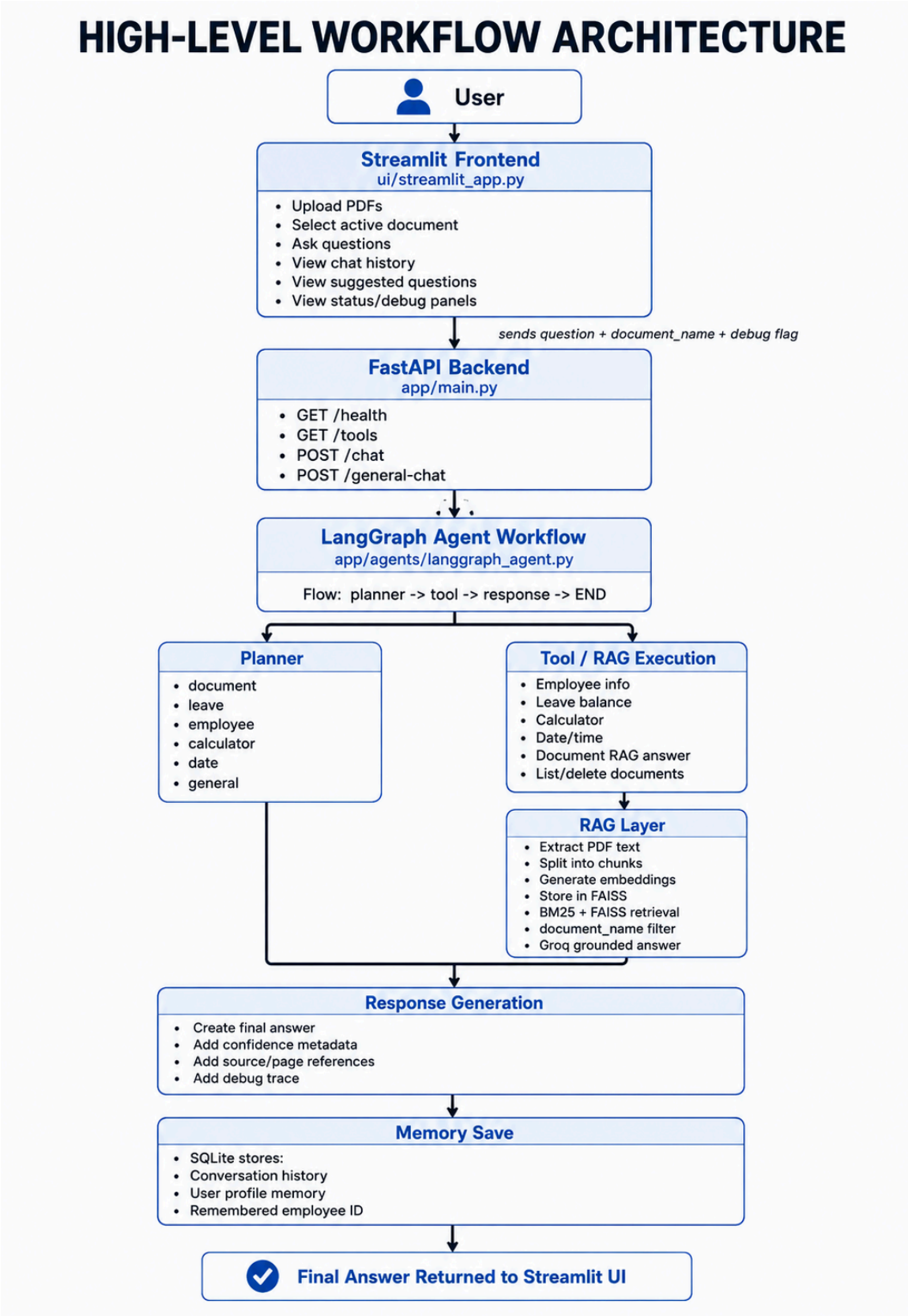
Instead of manually searching files or switching between multiple systems, the user can interact with one AI-powered assistant.

3. PROBLEM STATEMENT / NEED

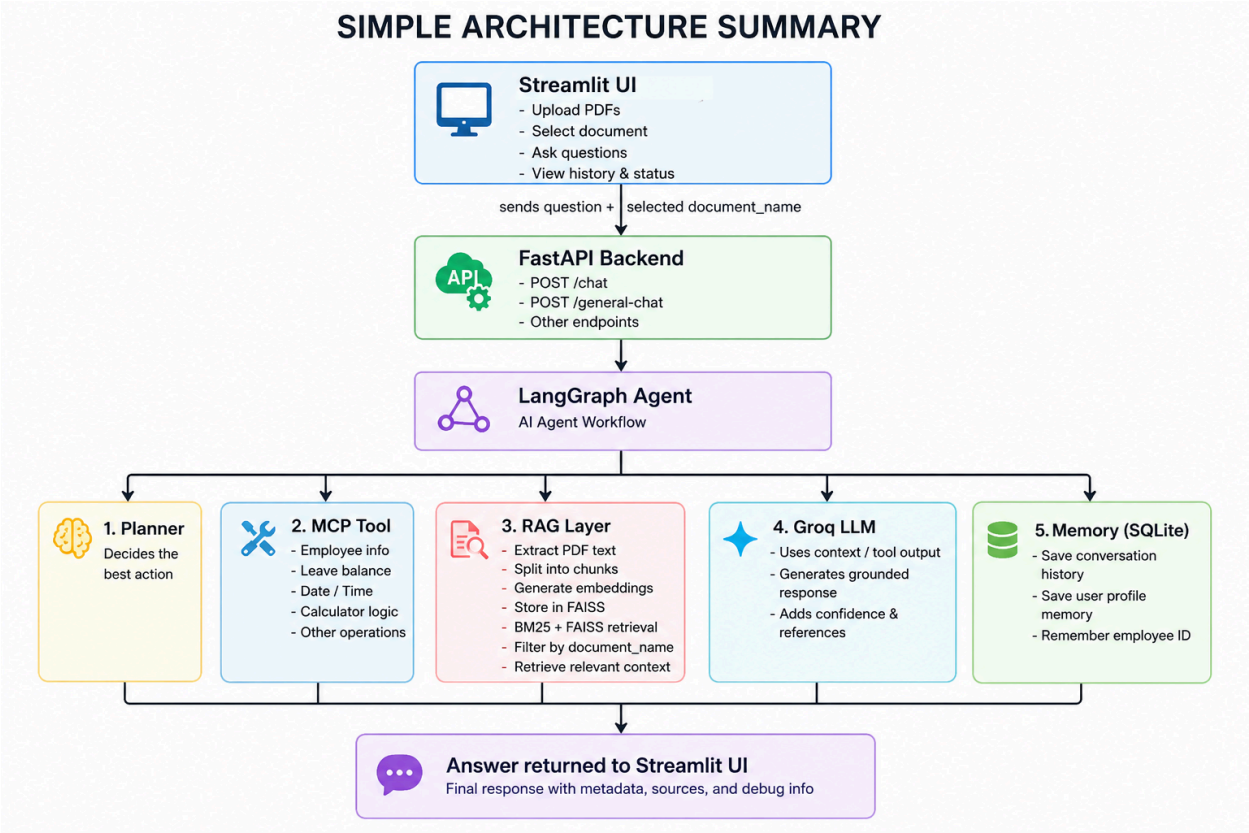
In many enterprises, information is spread across PDFs, HR records, internal databases, policy documents, and operational tools. Employees often waste time searching through long files, asking repetitive questions, or switching between systems.

This project solves that need by:
- Giving quick answers from uploaded documents.
- Reducing manual PDF search.
- Supporting structured enterprise data and unstructured documents.
- Keeping document answers scoped to the selected file.
- Providing a chat-based interface for enterprise tools.
- Making the system transparent with debug traces.

4. HIGH-LEVEL WORKFLOW ARCHITECTURE



5. SIMPLE ARCHITECTURE SUMMARY



6. LAYER 1: FRONTEND - STREAMLIT

The user interacts with the system through the Streamlit UI.

Main file:
ui/streamlit_app.py

Main responsibilities:
- Upload PDF documents.
- Save uploaded PDFs into data/documents.
- Select the active document.
- Ask document-specific or general questions.
- Display suggested questions from the selected document.
- Send chat requests to the backend.
- Show chat history.
- Show source and page references.
- Show confidence metadata.
- Show backend, RAG, document, and MCP status.
- Show compact and advanced debugger traces.

The Streamlit frontend acts as the user-facing layer. It does not directly generate AI responses. Instead, it sends user questions and selected document information to the FastAPI backend.

7. LAYER 2: BACKEND API - FASTAPI

FastAPI acts as the backend service between the Streamlit frontend and the LangGraph AI workflow.

Main file:
app/main.py

Important endpoints:
- GET /health
- GET /tools
- POST /chat
- POST /general-chat

Example request sent by Streamlit:

```
{
  "question": "Summarize this resume",
  "document_name": "SACHIN_CV0.pdf",
  "debug": true
}
```

Main responsibilities:
- Receive user questions from Streamlit.
- Receive active document name.
- Pass the request to the LangGraph workflow.
- Return the final answer.
- Return debug traces when debug mode is enabled.
- Return system health information.
- Return available tool information.

8. LAYER 3: AGENT WORKFLOW - LANGGRAPH

LangGraph controls the main AI workflow.

Main file:
app/agents/langgraph_agent.py

Workflow:
planner -> tool -> response -> END

Each question goes through this process:
1. Receive the user query.
2. Send the query to the planner.
3. Decide the query type.

4. Run the correct tool or RAG path.
5. Generate the final response.
6. Save conversation memory.
7. Return the answer to FastAPI.

LangGraph makes the architecture modular because planning, tool execution, response generation, and memory save are separated into clear workflow steps.

9. LAYER 4: PLANNER

The planner classifies the user query into one action.

Main file:
app/agents/planner.py

Possible actions:
- document
- leave
- employee
- calculator
- date
- general

Example routing:
- "What is my leave balance?" -> leave
- "Summarize this PDF" -> document
- "What is today's date?" -> date
- "Who is EMP101?" -> employee
- "10 + 25" -> calculator
- "Explain RAG" -> general

Planner modes:
1. Keyword-based planner
   - Default mode.
   - Uses keywords and rules to classify the query.

2. Optional LLM-based planner
   - Can use Groq for query classification.
   - Enabled when USE_LLM_PLANNER=true.
   - Falls back to keyword routing if the model call fails or the API key is missing.

10. LAYER 5: TOOL / MCP LAYER

The project uses an in-process MCP-style tool system. This is not a separate networked MCP server. It is a local tool gateway inside the application.

Main files:
app/mcp/mcp_server.py
app/tools/enterprise_tools.py
app/tools/calculator.py
app/tools/date_tool.py

Tools include:
- Employee info
- Leave balance
- Calculator
- Date/time
- Document answer
- List documents
- Delete document
- Rebuild index

Purpose:
The tool layer separates enterprise logic from the agent workflow. This makes the system easier to extend with real HRMS, CRM, ERP, ticketing systems, or database connectors in the future.

11. LAYER 6: RAG LAYER

For document questions, the system uses RAG, which means Retrieval-Augmented Generation.

Main files:
app/rag/ingest.py
app/rag/retriever.py
app/rag/suggestions.py

RAG workflow diagram:

PDF Upload
  |
  v
Extract Text
  |
  v
Split Text into Chunks
  |
  v
Generate Embeddings
  |

v
Store Embeddings in FAISS
|
v
Retrieve Relevant Chunks
|
v
Apply BM25 + FAISS Hybrid Ranking
|
v
Apply document_name Filter
|
v
Send Context to Groq LLM
|
v
Generate Grounded Answer with Source/Page References

Technologies used:
- FAISS for vector search.
- sentence-transformers/all-MiniLM-L6-v2 for embeddings.
- BM25 for keyword ranking.
- Groq LLM for final response generation.
- document_name filtering to answer only from the selected PDF.

Why document_name filtering matters:
When multiple PDFs are uploaded, the system must avoid mixing information from different files. The selected document name is sent with the chat request and used during retrieval, so answers are generated only from the active PDF.

Example:
If the user asks:
"Summarize this resume"

and the active document is:
SACHIN_CV0.pdf

then the backend receives:
```
{
  "question": "Summarize this resume",
  "document_name": "SACHIN_CV0.pdf"
}
```

The retriever only searches chunks from SACHIN_CV0.pdf, preventing wrong-source answers.

12. LAYER 7: MEMORY LAYER

The memory layer stores conversation history and user profile information.

Main files:
app/memory/memory.py
app/database/database.py

Databases:
data/enterprise.db
data/memory.db

Stored information:
- Chat history
- User conversations
- Remembered employee ID
- User profile memory
- Sample employee data
- Leave balance data

Database purpose:
- data/enterprise.db stores sample enterprise data such as employees and leave balance.
- data/memory.db stores conversations and remembered user profile values.

Example sample data:
Employee ID: EMP101
Name: Sachin
Department: AI Team
Casual leave: 8
Sick leave: 5

13. END-TO-END REQUEST FLOW EXAMPLE

User question:
Summarize this resume

Active document:
SACHIN_CV0.pdf

Request sent by Streamlit:
```
{
  "question": "Summarize this resume",
  "document_name": "SACHIN_CV0.pdf",
  "debug": true
}
```

Backend flow:
1. Streamlit sends the request to FastAPI.
2. FastAPI receives the request at POST /chat.

3. FastAPI calls the LangGraph agent workflow.
4. LangGraph sends the query to the planner.
5. Planner classifies the query as document.
6. The tool node calls the document RAG tool.
7. RAG retrieves chunks only from SACHIN_CV0.pdf.
8. BM25 + FAISS hybrid ranking selects the best chunks.
9. Retrieved context is sent to Groq.
10. Groq generates a grounded answer using only the supplied context.
11. SQLite saves the user question and final answer.
12. FastAPI returns the answer and debug trace to Streamlit.
13. Streamlit displays the final answer with confidence, source pages, and debug details.

14. TECHNOLOGY STACK

Frontend: Streamlit
Backend: FastAPI
Agent Workflow: LangGraph
Planner: Keyword planner with optional Groq LLM planner
Tool Layer: MCP-style in-process tools
Vector Database: FAISS
Keyword Search: BM25
Embeddings: sentence-transformers/all-MiniLM-L6-v2
LLM: Groq llama-3.3-70b-versatile
Database: SQLite
Language: Python

15. PROJECT FILE STRUCTURE

Enterprise-AI-Operations-Copilot/
|
+-- app/
| |
| +-- agents/
| | +-- langgraph_agent.py
| | +-- planner.py
| |
| +-- database/
| | +-- database.py
| |
| +-- llm/
| | +-- general.py
| |
| +-- mcp/
| | +-- mcp_server.py
| |
| +-- memory/

```
|   |   +-- memory.py
|   |
|   +-- rag/
|   |   +-- ingest.py
|   |   +-- retriever.py
|   |   +-- suggestions.py
|   |
|   +-- tools/
|   |   +-- enterprise_tools.py
|   |   +-- calculator.py
|   |   +-- date_tool.py
|   |
|   +-- main.py
|
+-- ui/
|   +-- streamlit_app.py
|
+-- data/
|   +-- documents/
|   +-- faiss_index/
|   +-- enterprise.db
|   +-- memory.db
|
+-- screenshots/
+-- README.md
+-- WORKFLOW.md
+-- PROJECT_DOCUMENTATION.md
+-- requirements.txt
+-- .env
```

16. ENVIRONMENT VARIABLES

Example .env file:

```
GROQ_API_KEY=your_groq_api_key_here
USE_LLM_PLANNER=false
COPILOT_API_URL=http://127.0.0.1:8000
PLANNER_MODEL=optional_planner_model
```

Variable purpose:
- GROQ_API_KEY: Required for Groq LLM responses.
- USE_LLM_PLANNER: Enables optional LLM-based query planning.
- COPILOT_API_URL: Sets the backend URL used by Streamlit.
- PLANNER_MODEL: Optional planner model override.

Important:
Never upload the .env file to GitHub.

17. HOW TO RUN THE PROJECT

Step 1: Create a virtual environment

```
python -m venv venv
source venv/bin/activate
```

For Windows:
```
venv\Scripts\activate
```

Step 2: Install dependencies

```
pip install -r requirements.txt
```

Step 3: Create a .env file

```
GROQ_API_KEY=your_groq_api_key_here
USE_LLM_PLANNER=false
COPILOT_API_URL=http://127.0.0.1:8000
```

Step 4: Start the FastAPI backend

```
uvicorn app.main:app --reload
```

Step 5: Start the Streamlit frontend in another terminal

```
streamlit run ui/streamlit_app.py
```

18. EXAMPLE USE CASES

Document query:
User: Summarize this PDF.
Action: document

Leave query:
User: What is my leave balance?
Action: leave

Employee query:
User: Who is EMP101?
Action: employee

Date query:
User: What is today's date?
Action: date

Calculator query:
User: 25 + 75
Action: calculator

General query:
User: Explain what RAG means.
Action: general

19. KEY STRENGTHS

- Complete end-to-end working AI assistant.
- Combines structured enterprise data and unstructured PDF documents.
- Uses RAG for document-grounded answers.
- Uses hybrid BM25 + FAISS retrieval.
- Supports active document filtering.
- Prevents answers from mixing multiple PDFs.
- Provides source and page references.
- Shows confidence metadata for document answers.
- Includes planner, tool, RAG, and memory debug traces.
- Uses SQLite for local memory and enterprise sample data.
- Modular architecture with separate UI, API, planner, tools, RAG, and memory layers.
- Can be extended with real enterprise systems in the future.

20. FUTURE ENHANCEMENTS

- Add user authentication.
- Add role-based access control.
- Connect with real HRMS, CRM, ERP, or ticketing systems.
- Replace SQLite with PostgreSQL for production use.
- Add multi-user conversation history.
- Add document-level permissions.
- Support DOCX, Excel, PPT, and scanned PDFs.
- Add OCR for image-based documents.
- Improve chunking and reranking for better RAG accuracy.
- Add streaming responses in the UI.
- Add Docker deployment.
- Add cloud deployment support.
- Add audit logs for enterprise compliance.
- Add voice input and output.
- Add feedback buttons for answer quality.
- Add analytics dashboard for common questions and usage trends.

21. INTERVIEW EXPLANATION

The architecture is modular and layered. Streamlit handles the frontend and user interaction. FastAPI works as the backend API layer. LangGraph manages the agent workflow. The planner decides what type of query the user asked. MCP-style tools handle enterprise operations such as employee information, leave balance, calculator, and date/time.

For document questions, the RAG layer retrieves context from uploaded PDFs using FAISS vector search and BM25 keyword ranking. The selected document name is passed through the request flow so the answer stays scoped to the correct PDF. Groq generates the final answer, and SQLite stores enterprise sample data and conversation memory.

This separation makes the system easier to debug, extend, and connect with real enterprise systems in the future.

22. ONE-MINUTE INTERVIEW ANSWER

This project is an Enterprise AI Operations Copilot built using Streamlit, FastAPI, LangGraph, FAISS, SQLite, and Groq LLM.

It helps users ask operational and document-based questions from a single chat interface. Users can upload PDFs, select an active document, ask questions, check employee information, view leave balance, perform calculations, get date/time answers, and ask general AI queries.

The backend uses FastAPI and LangGraph to route each query. The planner classifies the query into actions such as document, leave, employee, calculator, date, or general. For document questions, the system uses a RAG pipeline with MiniLM embeddings, FAISS vector search, and BM25 hybrid ranking. The selected document name is passed through the request flow so answers stay scoped to the correct PDF.

SQLite is used for employee data, leave balance, conversation memory, and user profile memory. The Streamlit UI also includes a debugger panel that shows planner, tool, RAG, and memory events.

The main goal of this project is to reduce manual searching across enterprise documents and tools by giving users one AI-powered operations assistant.

User Interface:

- - - - - - - - -